Mars CLI 애플리케이션 구조 분석

개요

`cmd/marsd` 디렉토리는 Mars 블록체인의 메인 CLI 애플리케이션을 구현합니다. 이 애플리케이션은 Cosmos SDK의 표준 CLI 패턴을 따르며, 노드 실행, 트랜잭션 처리, 쿼리 실행, 키 관리 등의 기능을 제공합니다.

디렉토리 구조

```
cmd/marsd/
├── main.go # CLI 애플리케이션 진입점
└── cmd/
├── root.go # 루트 명령어와 의존성 주입 설정
├── commands.go # 하위 명령어 등록과 구현
├── config.go # 설정 파일 초기화 및 커스터마이징
├── testnet.go # 단일 노드 테스트넷 명령어
└── testnet_multi_node.go # 다중 노드 테스트넷 명령어
```

1. 메인 애플리케이션 진입점 (main.go)

주요 기능

- CLI 애플리케이션의 시작점 역할

- 루트 명령어 초기화 및 실행
- 에러 처리 및 프로그램 종료 관리

실행 흐름

1. `cmd.NewRootCmd()` 를 통한 루트 명령어 생성
2. `svrcmd.Execute()` 를 통한 명령어 실행
3. 환경 변수 prefix와 기본 홈 디렉토리 설정
4. 에러 발생 시 적절한 종료 코드로 프로그램 종료

2. 루트 명령어 구조 (root.go)

의존성 주입 시스템

- `depinject` 패키지를 사용한 자동 의존성 해결
- 앱 설정, 로거, 클라이언트 컨텍스트 주입
- AutoCLI 옵션과 모듈 기본 관리자 초기화

클라이언트 컨텍스트 설정

```
// 클라이언트 컨텍스트 초기화 과정
clientCtx = clientCtx.WithCmdContext(cmd.Context()).WithViper(app.Name)
clientCtx, err := client.ReadPersistentCommandFlags(clientCtx, cmd.Flags())
clientCtx, err = config.ReadFromClientConfig(clientCtx)
```

PersistentPreRunE 함수의 역할

1. **출력 스트림 설정**: 표준 출력과 에러 스트림 설정
2. **클라이언트 컨텍스트 초기화**: 명령어 컨텍스트와 Viper 설정 추가
3. **플래그 읽기**: `--home`, `--chain-id` 등의 지속적 플래그 처리
4. **설정 파일 로딩**: `client.toml` 파일에서 클라이언트 설정 읽기
5. **서버 설정 처리**: `app.toml` 과 `config.toml` 파일 인터셉트 및 처리

IBC 모듈 등록

- IBC 모듈들은 아직 의존성 주입을 지원하지 않음
- 수동으로 모듈을 기본 모듈 관리자에 등록
- AutoCLI 옵션에도 모듈 추가하여 자동 CLI 생성 지원

3. 명령어 등록 및 구현 (commands.go)

initRootCmd 함수의 역할

루트 명령어에 모든 하위 명령어를 등록합니다:

기본 명령어들

- `genutilcli.InitCmd()` : 제네시스 파일 초기화
- `debug.Cmd()` : 디버깅 도구
- `confixcmd.ConfigCommand()` : 설정 파일 관리
- `pruning.Cmd()` : 상태 프루닝 관리
- `snapshot.Cmd()` : 스냅샷 관리

서버 명령어들

- `server.AddCommandsWithStartCmdOptions()` : 노드 시작 명령어 추가

- `server.StatusCommand()` : 노드 상태 조회
- `queryCommand()` : 쿼리 명령어 그룹
- `txCommand()` : 트랜잭션 명령어 그룹
- `keys.Commands()` : 키 관리 명령어

쿼리 명령어 구조

```go
func queryCommand() *cobra.Command {
cmd := &cobra.Command{
Use: "query",
Aliases: []string{"q"},
Short: "Querying subcommands",
// ... 기타 설정
}
// 하위 쿼리 명령어들 추가
cmd.AddCommand(
rpc.WaitTxCmd(), // 트랜잭션 대기
rpc.ValidatorCommand(), // 검증자 정보 조회
server.QueryBlockCmd(), // 블록 정보 조회
authcmd.QueryTxsByEventsCmd(), // 이벤트별 트랜잭션 조회
// ... 기타 쿼리 명령어들
)
return cmd
```

```
}
```

트랜잭션 명령어 구조

```
func txCommand() *cobra.Command {
cmd := &cobra.Command{
Use: "tx",
Short: "Transactions subcommands",
// ... 기타 설정
}
// 하위 트랜잭션 명령어들 추가
cmd.AddCommand(
authcmd.GetSignCommand(), // 트랜잭션 서명
authcmd.GetBroadcastCommand(), // 트랜잭션 브로드캐스트
authcmd.GetEncodeCommand(), // 트랜잭션 인코딩
authcmd.GetDecodeCommand(), // 트랜잭션 디코딩
// ... 기타 트랜잭션 명령어들
)
return cmd
}
```

애플리케이션 생성 함수들

- `newApp()` : 새로운 애플리케이션 인스턴스 생성

- `appExport()` : 애플리케이션 상태 내보내기

4. 설정 파일 관리 (config.go)

CometBFT 설정 초기화

```
func initCometBFTConfig() *cmtcfg.Config {
cfg := cmtcfg.DefaultConfig()
// 메모리 사용량이 높은 설정들 (주석 처리됨)
// cfg.P2P.MaxNumInboundPeers = 100
// cfg.P2P.MaxNumOutboundPeers = 40
return cfg
}
```

애플리케이션 설정 초기화

```
func initAppConfig() (string, interface{}) {
// 커스텀 앱 설정 구조체 정의
type CustomAppConfig struct {
serverconfig.Config `mapstructure:",squash"`
}
// 기본 서버 설정 로드
srvCfg := serverconfig.DefaultConfig()
// 최소 가스 가격 설정 (주석 처리됨)
```

```
// srvCfg.MinGasPrices = "0stake"

customAppConfig := CustomAppConfig{

Config: *srvCfg,

}

// 기본 설정 템플릿 사용

customAppTemplate := serverconfig.DefaultConfigTemplate

return customAppTemplate, customAppConfig

}
```

5. 테스트넷 명령어들

단일 노드 테스트넷 (testnet.go)

NewInPlaceTestnetCmd 함수

- 로컬 메인넷 노드를 테스트넷으로 변환
- 기존 검증자 데이터를 새로운 검증자 정보로 교체
- 테스트용 계정에 자금 지급

주요 기능들

1. **검증자 설정**: 새로운 검증자 생성 및 등록
2. **스테이킹 설정**: 검증자 파워와 위임 설정
3. **배포 설정**: 배포 보상과 수수료 초기화
4. **슬래싱 설정**: 검증자 서명 정보 설정
5. **뱅크 설정**: 테스트 계정에 토큰 지급

다중 노드 테스트넷 (testnet_multi_node.go)

NewTestnetMultiNodeCmd 함수

- 여러 검증자로 구성된 테스트넷 초기화
- Docker Compose 등을 통한 분산 실행 지원
- 각 노드별 개별 설정 파일 생성

초기화 과정

1. **노드 디렉토리 생성**: 각 검증자별 개별 디렉토리
2. **키 생성**: 각 노드별 검증자 키와 계정 키 생성
3. **제네시스 트랜잭션 생성**: 각 검증자의 gentx 파일 생성
4. **설정 파일 생성**: 각 노드별 config.toml과 app.toml 생성
5. **피어 설정**: 노드 간 연결을 위한 persistent peers 설정

6. 설정 파일 로딩 및 검증 로직

클라이언트 설정 로딩 과정

1. **Viper 초기화**: 환경 변수와 설정 파일 읽기 준비
2. **플래그 처리**: 명령줄 플래그에서 설정 값 읽기
3. **설정 파일 읽기**: `client.toml` 에서 클라이언트 설정 로드
4. **검증**: 설정 값의 유효성 검사
5. **컨텍스트 업데이트**: 클라이언트 컨텍스트에 설정 적용

서버 설정 인터셉트

```
// 서버 설정 파일들을 인터셉트하고 처리
return server.InterceptConfigsPreRunHandler(
cmd,
customAppTemplate, // 앱 설정 템플릿
customAppConfig, // 앱 설정 객체
customCMTConfig // CometBFT 설정
)
```

설정 파일 종류

1. **client.toml**: 클라이언트 설정 (체인 ID, 노드 주소, 키링 백엔드)
2. **app.toml**: 애플리케이션 설정 (API, gRPC, 최소 가스 가격)
3. **config.toml**: CometBFT 설정 (P2P, RPC, 합의 매개변수)

7. 에러 처리 및 검증

의존성 주입 에러 처리

```
if err := depinject.Inject(...); err != nil {
panic(err) // 의존성 주입 실패 시 패닉
}
```

설정 로딩 에러 처리

```
clientCtx, err := client.ReadPersistentCommandFlags(clientCtx, cmd.Flags())
if err != nil {
return err // 에러를 상위로 전파
}
```

테스트넷 초기화 에러 처리

```
func handleErr(err error) {
if err != nil {
fmt.Fprintln(os.Stderr, err.Error())
os.Exit(1) // 에러 발생 시 프로그램 종료
}
}
```

8. AutoCLI 기능

자동 CLI 생성

- 각 모듈의 메시지와 쿼리에 대한 CLI 명령어 자동 생성
- 프로토콜 버퍼 정의를 기반으로 한 명령어 구조 생성
- 개발자가 수동으로 CLI 명령어를 작성할 필요성 감소

AutoCLI 옵션 설정

```
if err := autoCliOpts.EnhanceRootCommand(rootCmd); err != nil {
panic(err)
}
```

9. 명령어 실행 흐름

전체 실행 과정

1. **main() 함수 시작**: 프로그램 진입점
2. **루트 명령어 생성**: `NewRootCmd()` 호출
3. **의존성 주입**: 필요한 컴포넌트들 초기화
4. **명령어 등록**: 모든 하위 명령어들 등록
5. **AutoCLI 적용**: 자동 생성된 명령어들 추가
6. **명령어 실행**: `svrcmd.Execute()` 호출
7. **PersistentPreRunE 실행**: 모든 명령어 실행 전 공통 초기화
8. **실제 명령어 실행**: 사용자가 지정한 명령어 실행
9. **결과 반환**: 성공 또는 에러 코드로 프로그램 종료

이러한 구조를 통해 Mars 블록체인은 표준적이고 확장 가능한 CLI 인터페이스를 제공하며, 개발자와 사용자 모두에게 일관된 경험을 제공합니다.